CHAPTER 2. THEORETICAL BACKGROUND

2.2. Convolutional Neural Networks (CNNs)

2.2.1. Overview and Basic Architecture of CNNs

Mạng thần kinh tích chập (Convolutional Neural Networks - CNNs) đại diện cho một lớp kiến trúc học sâu (deep learning) có khả năng trích xuất đặc trưng cực kỳ mạnh mẽ, được thiết kế chuyên biệt để xử lý và phân tích các loại dữ liệu có cấu trúc lưới không gian, điển hình như hình ảnh kỹ thuật số hai chiều[1]. Khác biệt cốt lõi tạo nên sự thành công của CNN so với các mạng nơ-ron đa tầng kết nối đầy đủ (Fully Connected Networks - FCN) truyền thống nằm ở việc mô hình này mô phỏng cơ chế hoạt động của vỏ não thị giác sinh học. Thay vì kết nối mọi nơ-ron ở một tầng với toàn bộ nơ-ron ở tầng kế tiếp, CNN tận dụng cấu trúc tô-pô của dữ liệu thông qua ba nguyên lý thiết kế nền tảng: tương tác cục bộ (local interactions hay sparse connectivity), chia sẻ tham số (parameter sharing), và các biểu diễn bất biến theo vị trí dịch chuyển (equivariant representations)[1].

Kiến trúc tổng thể của một mạng CNN truyền thống là sự kết hợp có hệ thống giữa các tầng tích chập (Convolutional layers) để tìm kiếm đặc trưng, các tầng hàm kích hoạt (Activation layers) để tạo tính phi tuyến, các tầng gom nhóm (Pooling layers) nhằm nén thông tin, và kết thúc bằng các tầng kết nối đầy đủ (Fully Connected layers) để đưa ra quyết định phân loại hoặc dự đoán[1].

2.2.2. Convolutional Layer

Tầng tích chập (Convolutional Layer) đóng vai trò là động cơ cốt lõi của mạng CNN, chịu trách nhiệm chính trong việc thực hiện các phép biến đổi toán học nhằm tạo ra các bản đồ đặc trưng (feature maps) từ dữ liệu thô[1]. Về mặt toán học, phép tích chập trên một không gian ảnh hai chiều được định nghĩa là sự tích lũy chập của một ma trận ảnh đầu vào I và một hạt nhân (kernel) hay bộ lọc (filter) K. Phương trình tổng quát được biểu diễn dưới dạng:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n)$$

Quá trình trượt bộ lọc qua toàn bộ không gian của ma trận đầu vào cho phép mạng học được các biểu diễn mang tính không gian sâu sắc. Mỗi nơ-ron trong bản đồ đặc trưng đầu ra chỉ tiếp nhận thông tin từ một khu vực không gian nhỏ của đầu vào, qua đó khai thác giả định rằng các điểm ảnh lân cận thường có mối tương quan mạnh mẽ và mang ý nghĩa ngữ nghĩa mật thiết hơn so với các điểm

ảnh ở cách xa nhau[1]. Cùng với đó, việc chia sẻ cùng một ma trận trọng số của bộ lọc trên toàn bộ không gian ảnh không chỉ giúp giảm hàm số lượng tham số một cách triệt để, ngăn chặn hiện tượng quá khớp (overfitting) do số chiều dữ liệu quá lớn, mà còn mang lại đặc tính bất biến dịch chuyển. Hệ quả là, nếu một đối tượng mục tiêu xuất hiện ở bất kỳ tọa độ nào trong ảnh, bộ lọc đó vẫn sở hữu khả năng kích hoạt và nhận diện chính xác[1].
Để kiểm soát kích thước không gian của bản đồ đặc trưng và mở rộng trường tiếp nhận một cách linh hoạt, phép tích chập được tinh chỉnh thông qua các siêu tham số cơ bản, được tóm tắt trong Bảng 1. Thêm khái niệm channel, feature map

Siêu tham số	Mô tả cơ chế hoạt động	Tác động đến bản đồ đặc trưng đầu ra
Bước trượt (Stride)	Khoảng cách (tính bằng pixel) mà bộ lọc di chuyển sau mỗi lần tính toán phép nhân chập[1].	Bước trượt lớn ($S >$) làm giảm nhanh kích thước không gian của ảnh đầu ra, tối ưu hóa tốc độ tính toán nhưng có thể làm mất thông tin cục bộ[1].
Chèn lót (Padding)	Bổ sung các dải pixel (thường có giá trị 0) bao quanh đường viền của ma trận đầu vào trước khi chập[1].	Giúp bảo toàn kích thước không gian của bản đồ đặc trưng, đồng thời ngăn chặn việc các thông tin ở vùng biên của ảnh bị bỏ qua trong quá trình trích xuất[1].

Kích thước của bản đồ đặc trưng đầu ra O đối với một đầu vào có kích thước cạnh W, kích thước bộ lọc K, mức độ chèn lót P và bước trượt S được tính toán chính xác thông qua công thức: $O = \lfloor \frac{W-K+2P}{S} \rfloor + 1$.

2.2.3. Activation Functions

Các hàm kích hoạt (Activation Functions) là yếu tố quyết định được chèn ngay sau các phép tính ma trận tuyến tính trong tầng tích chập nhằm phá vỡ tính

tuyến tính thuần túy. Nếu không có các hàm này, bất kể mạng nơ-ron có sâu đến đâu, nó cũng chỉ có thể biểu diễn tương đương với một mô hình hồi quy tuyến tính duy nhất. Các hàm kích hoạt cho phép mạng thần kinh học và xấp xỉ các đa tạp phi tuyến tính cực kỳ phức tạp trong không gian dữ liệu nhiều chiều[1].
Việc lựa chọn hàm kích hoạt phù hợp ảnh hưởng trực tiếp đến tốc độ hội tụ và khả năng truyền gradient của mạng. Bảng 2 so sánh ba loại hàm kích hoạt phổ biến nhất trong kiến trúc học sâu lịch sử và đương đại.

Hàm kích hoạt	Phương trình Toán học	Ưu điểm & Nhược điểm kỹ thuật
ReLU (Rectified Linear Unit)	$f(x) =$	**Ưu điểm:** Tốc độ tính toán vượt trội do chỉ là phép so sánh. Đạo hàm luôn bằng 1 khi $x >$, loại bỏ hiện tượng triệt tiêu đạo hàm, giúp mạng sâu hội tụ nhanh[1]. **Nhược điểm:** Hiện tượng "nơ-ron chết" nếu giá trị âm quá lớn đẩy đầu ra về 0 vĩnh viễn.
Sigmoid	$\sigma(x) =$	**Ưu điểm:** Cung cấp biểu diễn xác suất tự nhiên trong khoảng $(0, 1)$, mượt mà và dễ diễn giải[1]. **Nhược điểm:** Đạo hàm tiệm cận về 0 ở hai đầu mút, gây ra bão hòa

		(saturation) và triệt tiêu đạo hàm, luồng gradient bị dập tắt[1].
Tanh (Hyperbolic Tangent)	$\tanh(x) =$	**Ưu điểm:** Nén giá trị vào khoảng $(-1, 1)$, trung bình đầu ra gần với 0 giúp quá trình tối ưu hóa ổn định hơn Sigmoid[1]. **Nhược điểm:** Vẫn chịu chung rủi ro bão hòa đạo hàm khi giá trị đầu vào lớn[1].

Trong các mô hình CNN hiện đại, ReLU đã trở thành tiêu chuẩn mặc định ở hầu hết các tầng ẩn, trong khi Sigmoid thường chỉ được giữ lại ở tầng đầu ra của các bài toán phân loại nhị phân.

2.2.4. Pooling Layer

Tầng gom nhóm (Pooling Layer) thực hiện việc giảm lấy mẫu (down-sampling) kích thước không gian (chiều rộng và chiều cao) của các bản đồ đặc trưng sau khi chúng đi qua hàm kích hoạt, trong khi vẫn giữ nguyên số lượng kênh (channels)[1]. Sự hiện diện của tầng này mang lại ba lợi ích cốt lõi: giảm đáng kể khối lượng tính toán ở các tầng kế tiếp, kiểm soát số lượng tham số để hạn chế rủi ro quá khớp, và đặc biệt là tạo ra tính bất biến cục bộ đối với sự dịch chuyển (local translation invariance)[1]. Khi tính bất biến này được thiết lập, mạng nơ-ron có thể nhận diện một đặc điểm bất kể vị trí của nó có bị dịch chuyển vài pixel so với ảnh ban đầu.

Có hai phương pháp gom nhóm thống trị trong các kiến trúc CNN, được phân tích chi tiết trong Bảng 3.

Phương pháp Pooling	Cơ chế Hoạt động Toán học	Đặc điểm Ứng dụng

Max Pooling	Quét một cửa sổ (ví dụ $2 \times$) và chỉ giữ lại giá trị cực đại trong không gian đó[1].	Hiệu quả vượt trội trong việc bảo tồn các tín hiệu kích hoạt mạnh (như các cạnh, đường nét sắc sảo). Loại bỏ nhiễu nền yếu[1].
Average Pooling	Tính giá trị trung bình của toàn bộ các phần tử nằm trong vùng cửa sổ[1].	Bảo tồn thông tin ngữ cảnh tổng thể tốt hơn nhưng làm mờ các chi tiết nổi bật. Thường dùng ở dạng Global Average Pooling ở cuối mạng[1].

2.2.5. Fully Connected Layer and Softmax

Sau khi các đặc trưng cục bộ phức tạp được trích xuất và tinh lọc thông qua chuỗi các khối tích chập và gom nhóm xen kẽ, các bản đồ đặc trưng nhiều chiều (multi-dimensional tensors) sẽ được trải phẳng (flattened) thành các vector một chiều. Các vector này sau đó được truyền vào các tầng kết nối đầy đủ (Fully Connected layers - FC)[1]. Tại tầng FC, mạng lưới mất đi khái niệm về cấu trúc không gian 2D; thay vào đó, mọi nơ-ron đều được kết nối chéo với toàn bộ nơ-ron của tầng trước. Chức năng tối hậu của tầng này là tổng hợp, đối chiếu các đặc trưng bậc cao trên toàn bộ bức ảnh để tìm ra các mẫu logic (logic patterns) phục vụ cho quyết định phân loại cuối cùng[1].

Ở tầng đầu ra (output layer) trong các bài toán phân loại đa lớp (multi-class classification), việc sử dụng các giá trị tuyến tính thô (logits) là không đủ để diễn giải xác suất. Do đó, hàm Softmax được áp dụng nhằm chuyển đổi vector logits z thành một phân phối xác suất dự đoán P tuân thủ nghiêm ngặt các quy tắc tiên đề xác suất. Công thức tính xác suất cho lớp thứ i trong tổng số C lớp được định nghĩa là:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Quá trình này đảm bảo tổng xác suất của tất cả các lớp luôn bằng 1 ($\sum P(y_i) = 1$) và mỗi xác suất thành phần đều nằm trong khoảng không gian $(0, 1)$ [1]. Phân phối xác suất này chính là nền tảng cốt lõi để tính toán sai số trong các hàm mất mát.

2.2.6. Regularization via Batch Normalization and Dropout

Sự gia tăng độ sâu của các mạng nơ-ron kéo theo những rào cản nghiêm trọng trong quá trình tối ưu hóa. Hai kỹ thuật mang tính cách mạng là Batch Normalization và Dropout đã được phát triển để điều tiết luồng tín hiệu và duy trì sự ổn định của mạng.

Batch Normalization (Chuẩn hóa theo lô): Được định hình bởi Ioffe và Szegedy (2015), Batch Normalization (BN) ra đời để giải quyết vấn đề thay đổi phân phối nội tại (internal covariate shift)[7]. Hiện tượng này xảy ra khi phân phối dữ liệu đầu vào tại một lớp ẩn thay đổi liên tục qua từng vòng lặp do sự cập nhật trọng số của các lớp trước đó. Sự bất ổn này đẩy các giá trị kích hoạt vào vùng bão hòa của các hàm phi tuyến, dập tắt gradient và buộc các nhà nghiên cứu phải sử dụng tốc độ học (learning rate) cực kỳ nhỏ[7].

BN khắc phục vấn đề này bằng cách ép phân phối đầu vào của mỗi lớp phải được làm trắng (whitened) - tức là có trung bình bằng 0 và phương sai bằng 1, thực hiện độc lập trên từng lô nhỏ (mini-batch) trong quá trình huấn luyện[7]. Cụ thể, xét một lô nhỏ $\mathcal{B} = \{x_1, \dots, x_m\}$, quy trình chuẩn hóa biến đổi $\mathrm{BN}_{\gamma,\beta}(x_i)$ được tiến hành qua các phương trình toán học sau[8]:

1. Tính trung bình lô nhỏ: $\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^{m} x_i$
2. Tính phương sai lô nhỏ: $\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2$
3. Chuẩn hóa giá trị: $\widehat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$ (với ϵ là hằng số cực nhỏ đảm bảo sự ổn định khi chia).
4. Khôi phục năng lực biểu diễn thông qua phép biến đổi Affine: $y_i = \gamma \widehat{x}_i + \beta$.

Các tham số γ (độ giãn) và β (độ dịch chuyển) là những tham số có thể học (learnable parameters). Sự tồn tại của chúng đảm bảo rằng nếu việc chuẩn hóa làm mất đi thông tin hữu ích của dữ liệu, mạng hoàn toàn có thể tự học cách đảo ngược quá trình chuẩn hóa đó[8]. Khi chuyển sang giai đoạn kiểm thử (inference),

mô hình không còn dựa vào dữ liệu của lô hiện tại mà sử dụng kỳ vọng quần thể (population expectation) và phương sai không chệch (unbiased variance estimation) được tích lũy từ toàn bộ tập huấn luyện[8]. Biến đổi lúc này trở thành một phép tính tuyến tính tất định:

$$y = \frac{\gamma}{\sqrt{\mathrm{Var}[x] + \epsilon}} x + \left(\beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} \right)$$

Về mặt lý thuyết tối ưu hóa, BN giúp làm mịn đáng kể cảnh quan hàm mất mát (loss landscape), cải thiện tính chất Lipschitz của gradient[9]. Bằng cách giới hạn độ lớn của đạo hàm, gradient trở nên dễ dự đoán hơn, cho phép mô hình sử dụng tốc độ học lớn, tăng tốc độ hội tụ theo cấp số nhân và giảm thiểu sự phụ thuộc vào các kỹ thuật khởi tạo trọng số phức tạp[7].

Dropout (Kỹ thuật Bỏ mạng): Trong khi BN hỗ trợ quá trình tối ưu hóa, Dropout (được phát triển bởi Hinton, Krizhevsky và cộng sự) là một trong những phương pháp điều tiết (regularization) hiệu quả nhất để ngăn chặn sự quá khớp[10]. Trong thực nghiệm trên bộ dữ liệu ImageNet, Dropout được cấu hình để ngẫu nhiên ngắt (zeroing out) các nơ-ron ẩn (bao gồm mọi liên kết tiến và lùi) với xác suất $p = 0.5$ tại mỗi lượt truyền dữ liệu[11].

Cơ chế vô hiệu hóa ngẫu nhiên này phá vỡ hiện tượng đồng thích nghi phức tạp (complex co-adaptations) giữa các nơ-ron[10]. Thay vì một nhóm nơ-ron cùng nhau "ghi nhớ" một mẫu dữ liệu cụ thể, Dropout ép mỗi nơ-ron riêng lẻ phải tự học các đặc trưng khái quát, hữu ích độc lập, vì nó không thể tin tưởng hoàn toàn vào sự hiện diện của các nơ-ron kế cận. Để đảm bảo mô hình không bị bùng nổ trọng số, Dropout thường được kết hợp với việc đặt rào cản ràng buộc chuẩn L_2 cực đại lên các trọng số liên kết[11]. Tại thời điểm kiểm thử (test time), Dropout được tắt đi, toàn bộ mạng lưới hoạt động đồng thời. Để bù đắp cho lượng tín hiệu đầu ra lớn hơn gấp đôi so với khi huấn luyện, các trọng số liên kết sẽ được nhân với tỷ lệ $(1 - p)$ (tức là giảm đi một nửa), đảm bảo kỳ vọng phân phối tín hiệu được duy trì ổn định[11].

2.3. CNN Training and Optimization

2.3.1. Forward Propagation and Loss Function

Quá trình huấn luyện của mạng CNN vận hành dựa trên một chu trình khép kín. Bắt đầu với lan truyền tiến (Forward Propagation), ma trận điểm ảnh thô được

truyền liên tục qua các tầng tích chập, gom nhóm và kết nối đầy đủ để chuyển đổi thành một vector xác suất phân loại[1]. Để định lượng xem dự đoán này sai lệch bao nhiêu so với thực tế, mô hình yêu cầu một hàm mất mát (Loss Function).
Hàm mất mát hoạt động như một la bàn, định hướng cho toàn bộ quá trình học. Dựa vào bản chất của bài toán,hàm categorical croos-entropy được ưu tiên sử dụng dựa trên nền tảng lý thuyết thông tin (đo lường khoảng cách Kullback-Leibler giữa hai phân phối xác suất)[12]:

Categorical Cross-Entropy (CCE)	$\mathrm{L} =$	Dùng cho phân loại đa lớp (multi-class). Trong đó t_i là nhãn đúng mã hóa one-hot, p_i là xác suất dự đoán từ hàm Softmax. Chỉ trích xuất sai số từ lớp có nhãn thực, thúc đẩy xác suất của lớp đó tiến dần về 1[12].

Việc tối thiểu hóa hàm suy hao này đồng nghĩa với việc tối đa hóa khả năng xảy ra (likelihood) của dữ liệu dưới bộ tham số hiện tại.

2.3.2. Backpropagation and Gradient Descent

Khi sai số từ hàm mất mát được xác định, mạng lưới tiến hành lan truyền ngược (Backpropagation). Sử dụng quy tắc chuỗi (chain rule) của vi tích phân, hệ thống tính toán các đạo hàm riêng (gradients) của hàm mất mát đối với hàng triệu trọng số trong mạng, bắt đầu từ tầng cuối cùng lùi dần về tầng đầu tiên[1].
Các gradient này chỉ ra hướng đi dốc nhất trên bề mặt của không gian hàm mất mát. Phương pháp Gradient Descent (Hạ gradient) sẽ cập nhật bộ trọng số bằng cách di chuyển một bước nhỏ ngược chiều với gradient. Trong thực tiễn của học sâu, do tập dữ liệu huấn luyện thường chứa hàng triệu mẫu, việc tính toán gradient trên toàn bộ dữ liệu là bất khả thi. Do đó, Stochastic Gradient Descent (SGD) hoặc Mini-batch SGD được đưa vào áp dụng[1]. SGD lấy mẫu ngẫu nhiên một lô nhỏ dữ liệu để tính xấp xỉ gradient, việc này tạo ra các bước đi mang tính ngẫu nhiên (stochastic), mang lại động năng giúp mô hình thoát khỏi các điểm yên ngựa (saddle points) - vốn là những điểm chết phổ biến trong không gian đa chiều, đồng thời mang lại sự linh hoạt tính toán cao[1].

2.3.3. Optimization Algorithms: The Adam Optimizer

Nổi bật nhất và hiện là tiêu chuẩn vàng trong huấn luyện học sâu là thuật toán **Adam (Adaptive Moment Estimation)**, công bố bởi Kingma và Ba (2014)[15]. Adam tích hợp điểm mạnh của hai phương pháp tiền nhiệm: AdaGrad (tối ưu hóa tốt với các gradient thưa thớt) và RMSProp (điều chỉnh tốc độ học dựa trên trung bình trượt của bình phương gradient để xử lý các mục tiêu không tĩnh)[15]. Thuật toán duy trì và cập nhật liên tục hai trạng thái ước lượng cho mỗi tham số tại bước t:

- **Ước lượng moment bậc 1** (trung bình gradient có trọng số):
  $$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
- **Ước lượng moment bậc 2** (phương sai gradient chưa định tâm):
  $$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Trong đó, β_1 và β_2 là các tỷ lệ suy giảm lũy thừa, thường được thiết lập mặc định ở mức rất sát 1 ($\beta_1 = 0.9, \beta_2 = 0.999$)[16]. Tại thời điểm khởi tạo, m_0 và v_0 là các vector 0. Sự khởi tạo này tạo ra một độ chệch (bias) rất nặng nề về 0 trong những vòng lặp đầu tiên. Về mặt toán học chứng minh bởi Kingma và Ba, giá trị kỳ vọng của moment bậc 2 bị chệch theo hệ số: $\mathbb{E}[v_t] = \mathbb{E}[g_t^2] \cdot (1 - \beta_2^t)$ [16]. Để khắc phục triệt để sai số này, Adam áp dụng các phương trình hiệu chỉnh độ chệch (bias-correction):
\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$Cuối cùng, trọng số θ được cập nhật độc lập qua từng chiều không gian:$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}

Với α là tốc độ học và $\epsilon = 10^{-8}$ nhằm ngăn lỗi chia cho 0[16]. Lý thuyết tối ưu hóa chứng minh rằng Adam đạt được giới hạn biên hối tiếc (regret bound) ở mức $O(\log d\sqrt{T})$, một sự cải thiện lớn so với $O(\sqrt{dT})$ của các phương pháp không thích ứng[15]. Đặc tính bất biến trước quy mô của các gradient và khả năng tự động thực hiện ủ tốc độ học (step size annealing) khiến Adam hội tụ vô cùng nhanh và ổn định[15].

2.3.4. Overfitting and Regularization

Overfitting (Quá khớp) là trạng thái mà mô hình sở hữu độ phức tạp biểu diễn vượt quá nhu cầu thực tế của dữ liệu, khiến nó bắt đầu học thuộc lòng (memorize) cả các tín hiệu nhiễu thay vì chắt lọc quy luật tổng quát[1]. Triệu chứng rõ rệt nhất là độ chính xác rất cao trên tập huấn luyện nhưng suy giảm nghiêm trọng khi đối mặt với dữ liệu chưa từng gặp.
Để trấn áp quá khớp, ngoài Dropout, kỹ thuật Weight Decay (L2 Regularization) thường được kết hợp. Bằng cách bổ sung một lượng phạt (penalty term) tỷ lệ thuận với tổng bình phương của tất cả các trọng số vào hàm mất mát gốc $\left(\text{Loss} + \frac{\lambda}{2}\sum w^2\right)$, mạng lưới bị ép phải phân tán sự chú ý vào nhiều điểm ảnh và đặc trưng khác nhau thay vì đặt cược toàn bộ phán đoán vào một vài vùng thông tin cục bộ[1].

2.4. Model Evaluation Metrics

2.4.1. Confusion Matrix

Trong bối cảnh đánh giá hiệu năng mô hình phân loại, việc chỉ dựa vào một chỉ số duy nhất có thể gây ra những nhận định sai lầm. Do đó, Ma trận nhầm lẫn (Confusion Matrix) được sử dụng như một lăng kính toàn diện, đối chiếu trực tiếp giữa dự đoán của mạng CNN và nhãn thực tế (ground truth) của tập kiểm thử[17]. Đối với một hệ thống phân loại, ma trận này bóc tách kết quả thành bốn nhóm nền tảng:

- **True Positives (TP):** Các mẫu mang đặc tính tích cực và mô hình nhận diện chính xác chúng là tích cực.
- **True Negatives (TN):** Các mẫu mang đặc tính tiêu cực và mô hình phân loại đúng là tiêu cực.
- **False Positives (FP - Lỗi loại I):** Mô hình báo động sai, dự đoán một mẫu tiêu cực thành tích cực.
- **False Negatives (FN - Lỗi loại II):** Mô hình bỏ lọt, không nhận diện được một mẫu tích cực thực sự[17].

2.4.2. Precision, Recall, F1-score and Averages

Từ nền tảng của Ma trận nhầm lẫn, hệ thống các chỉ số định lượng nâng cao được thiết lập. Bảng 5 trình bày chi tiết các công thức toán học và ngữ cảnh ứng dụng của từng chỉ số.

Chỉ số Đánh giá	Công thức Toán học	Ngữ cảnh Ứng dụng Thực tiễn

Accuracy (Độ chính xác)	$\frac{TP+TN}{TP+TN+FP+FN}$	Cung cấp tỷ lệ các dự đoán đúng trên toàn bộ tập dữ liệu. Tuy nhiên, chỉ số này mất đi tính khách quan nếu tập dữ liệu bị mất cân bằng (ví dụ: lớp thiểu số chỉ chiếm 5%)[17].
Precision (Độ chuẩn xác)	$\frac{TP}{TP+FP}$	Định lượng tỷ lệ các dự đoán tích cực thực sự chính xác. Chỉ số này mang tính sống còn đối với các hệ thống yêu cầu giảm thiểu lỗi loại I, nơi chi phí của một dự đoán sai là không thể chấp nhận được[17].
Recall (Độ nhạy)	$\frac{TP}{TP+FN}$	Khả năng mô hình "bắt" và bao quát được toàn bộ các mẫu tích cực. Ưu tiên hàng đầu trong y tế hoặc an ninh, nơi việc bỏ lọt (lỗi loại II) mang lại rủi ro chí mạng[17].
F1-score	$2 \times$	Là trung bình hài hòa (harmonic mean) của Precision và Recall. Chỉ số này chỉ tiến đến 1 khi và chỉ khi cả hai giá trị kia đều cao, thiết lập tiêu chuẩn công bằng cho dữ liệu mất cân đối[17].

Cơ chế Tính toán Trung bình Đa lớp (Multiclass Averaging): Đối với các bài toán có quy mô từ 3 lớp trở lên, các chỉ số trên không thể được áp dụng trực tiếp mà phải thông qua cơ chế tổng hợp đa lớp, tích hợp trực tiếp trong thư viện như Scikit-Learn[18]:

- **Macro-average ('macro'):** Tính toán các chỉ số (F1, Precision, Recall) hoàn toàn độc lập cho từng lớp, sau đó lấy trung bình cộng không trọng số. Phương pháp này đảm bảo mọi lớp, kể cả các lớp thiểu số cực nhỏ, đều có trọng lượng tác động ngang bằng, giúp phát hiện sự yếu kém cục bộ của mô hình[18].
- **Micro-average ('micro'):** Gộp chung toàn bộ dữ liệu TP, TN, FP, FN từ mọi lớp trước khi đưa vào công thức tính toán. Phân tích thống kê chỉ ra rằng trong phân loại đa lớp thuần túy, micro-F1, micro-Precision và Accuracy sẽ luôn quy tụ về cùng một giá trị toán học[18].
- **Weighted-average ('weighted'):** Biến thể của macro-average, trong đó chỉ số của mỗi lớp được nhân với một trọng số tỷ lệ thuận với số lượng mẫu thực tế (support) của lớp đó trong tập dữ liệu. Trọng số này triệt tiêu sự thiên lệch đối với các lớp quá nhỏ[18].

2.4.3. Validation Loss and Early Stopping

Theo dõi sự biến thiên của hàm mất mát trên hai tập dữ liệu độc lập (Training Loss và Validation Loss) qua từng chu kỳ (epoch) là thước đo trực quan nhất về tính khái quát hóa. Trong kịch bản lý tưởng, cả hai giá trị này cùng hội tụ về mức thấp. Tuy nhiên, nếu Training Loss tiếp tục lao dốc không phanh nhưng Validation Loss bắt đầu có xu hướng tạo đáy và tăng ngược trở lại, đó là dấu hiệu không thể chối cãi của quá khớp[1]. Để bảo vệ mô hình, cơ chế Dừng sớm (Early Stopping) được kích hoạt: hệ thống tự động chấm dứt quá trình huấn luyện khi Validation Loss không còn ghi nhận sự cải thiện sau một khoảng thời gian kiên nhẫn (patience) nhất định, và khôi phục lại bộ tham số tối ưu nhất ở chu kỳ đáy[1].

2.5. Representative Modern CNN Architectures

Sự phát triển của Deep Learning được đánh dấu bởi những kiến trúc mạng mang tính đột phá về cấu trúc toán học và quy mô triển khai. Bảng 6 so sánh ba kiến trúc nền tảng định hình hệ sinh thái thị giác máy tính.

Kiến trúc	Năm ra mắt & Đóng góp cốt lõi	Cấu trúc Kỹ thuật & Đặc trưng

LeNet-5	1998 (Yann LeCun). Mở đường cho kỷ nguyên CNN hiện đại, tối ưu cho nhận dạng chữ số MNIST[4].	Thiết lập cấu trúc kinh điển: Tầng Convolution $\rightarrow$ Tầng Subsampling (Pooling) $\rightarrow$ Tầng Fully Connected[2]. Dù nông và đơn giản, triết lý thiết kế này định hình mọi mô hình sau đó[4].
VGG	2014 (Simonyan & Zisserman). Chứng minh chiều sâu (depth) là chìa khóa của học biểu diễn đa tầng[8].	Loại bỏ bộ lọc lớn, đồng nhất sử dụng bộ lọc siêu nhỏ 3×3 với bước trượt 1 kết hợp MaxPool $2 \times$. Phiên bản VGG16 cực kỳ nặng với khoảng 138 triệu tham số[8].
ResNet	2016 (Kaiming He). Phá vỡ rào cản suy thoái, đưa mô hình chạm ngưỡng hàng trăm tầng[23].	Chuyển đổi hệ hình học từ học hàm nguyên bản sang học hàm thặng dư $F(x) +$ thông qua các kết nối tắt (skip connections), giúp gradient không bị triệt tiêu[23].

2.6. Techniques for Improving CNN Performances

2.6.1. Transfer Learning

Học chuyển giao (Transfer Learning) giải quyết bài toán thiếu hụt dữ liệu bằng cách tận dụng tri thức (trọng số) mà một mạng nơ-ron sâu đã học được từ một bộ dữ liệu quy mô khổng lồ (điển hình như ImageNet), sau đó ứng dụng sang một tác vụ mới, hẹp hơn[23].

Trong công trình nghiên cứu tiên phong của Yosinski và cộng sự (2014) khảo sát về tính khái quát hóa và độ sâu chuyển giao, thực nghiệm được thiết kế bằng

cách chia tập ImageNet thành tập Base A và Base B (500 lớp/tập), tạo ra các mạng kiểm chứng A_nB (chuyển n lớp từ A sang B, đóng băng) và mạng kiểm soát B_nB (chuyển n lớp từ chính B sang B, đóng băng)[27]. Kết quả thực nghiệm đã định hình nền tảng phân tích đặc trưng của mạng nơ-ron, bóc tách hai quy luật quan trọng[27]:

1. **Khái quát (General) vs Cụ thể (Specific):** Tại lớp 1 và lớp 2, mọi bộ lọc tự động hội tụ về các đặc trưng khái quát cao độ (như các cạnh, bộ lọc Gabor, hay đốm màu), không phụ thuộc vào bộ dữ liệu. Nhưng càng đi sâu vào mạng, các đặc trưng này đặc hóa mạnh mẽ để phục vụ cho các lớp ảnh ban đầu. Chuyển giao các lớp sâu này sang tác vụ khác biệt (ví dụ: từ phân loại đồ vật nhân tạo sang sinh vật tự nhiên) làm giảm hiệu suất nghiêm trọng[27].
2. **Sự đồng thích nghi dễ vỡ (Fragile Co-adaptation):** Thông qua mạng kiểm soát B_nB, nghiên cứu phát hiện rằng khi mạng bị cắt ở các lớp giữa (như lớp 3, 4, 5), dù dữ liệu mục tiêu không thay đổi, hiệu suất vẫn giảm. Nguyên nhân là do các nơ-ron ở các tầng trung gian có tính đồng thích nghi mật thiết. Khi đóng băng nửa dưới và khởi tạo ngẫu nhiên nửa trên, sự tương tác phức tạp này bị phá vỡ[27]. Nghiên cứu cũng nhấn mạnh rằng khởi tạo ngẫu nhiên ở các tầng thấp trên dữ liệu khổng lồ dẫn đến mất khả năng hội tụ hoàn toàn[27].

Để vượt qua rào cản này, chiến lược Tinh chỉnh (Fine-tuning - mạng A_nB^+) được áp dụng. Thay vì khóa cứng toàn bộ trọng số chuyển giao, mô hình mở khóa các tầng trên, cho phép chúng cập nhật chậm rãi thông qua đạo hàm ngược để phù hợp với tác vụ mới. Việc khởi tạo mạng bằng trọng số tinh chỉnh này mang lại mức tăng trưởng hiệu năng khái quát hóa đáng ngạc nhiên, ổn định hơn nhiều so với việc bắt đầu từ đầu[27].

2.6.2. Data Augmentation and Mixup

Tăng cường dữ liệu (Data Augmentation) là một thủ pháp mở rộng đa dạng sinh học của tập dữ liệu huấn luyện, thông qua việc áp dụng các phép biến đổi không gian như cắt xén (cropping), lật đối xứng, xoay, và biến đổi nhiễu loạn cường độ màu[1]. Những thay đổi nhân tạo này buộc mạng nơ-ron phải bất biến đối với các yếu tố ngoại cảnh, gia tăng ranh giới vững chắc của hệ thống.

.

Nguồn trích dẫn

1. https://drive.google.com/open?id=1eklgbMzmSp-Ndg6QbgILGVK_0QvyxOiZLSNz5S_CXVg
2. Gradient-Based Learning Applied to Document Recognition, http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf
3. Image classification with no data? - Towards Data Science, https://towardsdatascience.com/image-classification-with-no-data-b44089f8dc28/
4. LeNet-5 Complete Architecture - Medium, https://medium.com/codex/lenet-5-complete-architecture-84c6d08215f9
5. Volume 2, Issue 1 Isabela State University Linker, https://isujournals.ph/index.php/ject/article/download/212/76
6. Sentiment Analysis of Online New Energy Vehicle Reviews - MDPI, https://www.mdpi.com/2076-3417/13/14/8176
7. Exponential convergence rates for Batch Normalization:, https://proceedings.mlr.press/v89/kohler19a/kohler19a.pdf
8. Batch Normalization: Accelerating Deep Network Training by ... - arXiv, https://arxiv.org/abs/1502.03167
9. Batch normalization - Wikipedia, https://en.wikipedia.org/wiki/Batch_normalization
10. Neural network training: The basics and beyond, https://slazebni.cs.illinois.edu/spring25/lec09_training.pdf
11. https://arxiv.org/abs/1207.0580
12. What Is Cross-Entropy Loss Function? - 365 Data Science, https://365datascience.com/tutorials/machine-learning-tutorials/cross-entropy-loss/
13. Top 8 Loss Functions Made Simple & How To Implement Them, https://spotintelligence.com/2023/09/25/loss-functions/
14. Categorical Cross-Entropy: Unraveling its Potentials in Multi-Class, https://medium.com/@vergotten/categorical-cross-entropy-unraveling-its-potentials-in-multi-class-classification-705129594a01
15. Adam: A Method for Stochastic Optimization - arXiv, https://arxiv.org/pdf/1412.6980
16. https://arxiv.org/abs/1412.6980
17. Batch Normalization — Fanout, https://fanout.sh/daily/2026-07-10-batch-normalization
18. https://scikit-learn.org/stable/modules/model_evaluation.html
19. VGG-Net Architecture Explained - Medium, https://medium.com/@siddheshb008/vgg-net-architecture-explained-71179310050f
20. Best deep CNN architectures and their principles: from AlexNet to, https://theaisummer.com/cnn-architectures/
21. Writing VGG from Scratch in PyTorch - DigitalOcean, https://www.digitalocean.com/community/tutorials/vgg-from-scratch-pytorch

22. Very Deep Convolutional Networks for Large-Scale Image, https://www.researchgate.net/publication/265385906_Very_Deep_Convolutional_Networks_for_Large-Scale_Image_Recognition
23. A Refresher on Batch (Re-)Normalization | Luminovo - Medium, https://medium.com/luminovo/a-refresher-on-batch-re-normalization-5e0a1e902960
24. Deep Residual Learning for Image Recognition, https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf
25. https://arxiv.org/abs/1512.03385
26. Deep Residual Learning for Image Recognition, https://www.computer.org/csdl/proceedings-article/cvpr/2016/8851a770/12OmNxvwoXv
27. How transferable are features in deep neural networks? - arXiv, https://arxiv.org/abs/1411.1792
28. (PDF) How transferable are features in deep neural networks?, https://www.researchgate.net/publication/268079628_How_transferable_are_features_in_deep_neural_networks
29. How transferable are features in deep neural networks?, https://www.researchgate.net/publication/319770381_How_transferable_are_features_in_deep_neural_networks
30. [1710.09412] mixup: Beyond Empirical Risk Minimization - arXiv, https://arxiv.org/abs/1710.09412
31. Nonlinear Mixup: Out-Of-Manifold Data Augmentation for Text, https://ojs.aaai.org/index.php/AAAI/article/view/5822/5678
32. (PDF) Understanding Mixup Training Methods - ResearchGate, https://www.researchgate.net/publication/328009804_Understanding_Mixup_Training_Methods
33. Improved Mixed-Example Data Augmentation - arXiv, https://arxiv.org/html/1805.11272v4